

【专栏一：AI基础05】 - 【一张图讲清楚Function Call完整流程】



快乐飞猪ing



· 码龄7年 · 粉丝可见 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-27 11:55:07 · 641 阅读



AI 写同款 · GEO 优化

通义千问大模型之 *Function call* 实战

【代码】通义千问大模型之 *Function call* 实战。

[阅读详情 >](#)

前言

很多人第一次接触 Function Call 时，都会下意识地以为：

- 大模型既然知道要查天气、查快递、查订单，那它是不是就能自己直接去执行这些工具？

但真实情况并不是这样。[大模型](#) 本身擅长的是理解语言、生成内容、做决策，它并不会真的自己去点按钮、调用接口或者操作数据库。

在 Function Call 流程里，大模型真正做的事情，其实分成两次：

- 第一次：理解用户问题，看有哪些工具可用，判断要不要调用工具，并生成工具调用意图
- 第二次：接收工具执行结果，再组织成用户最终看到的自然语言回答

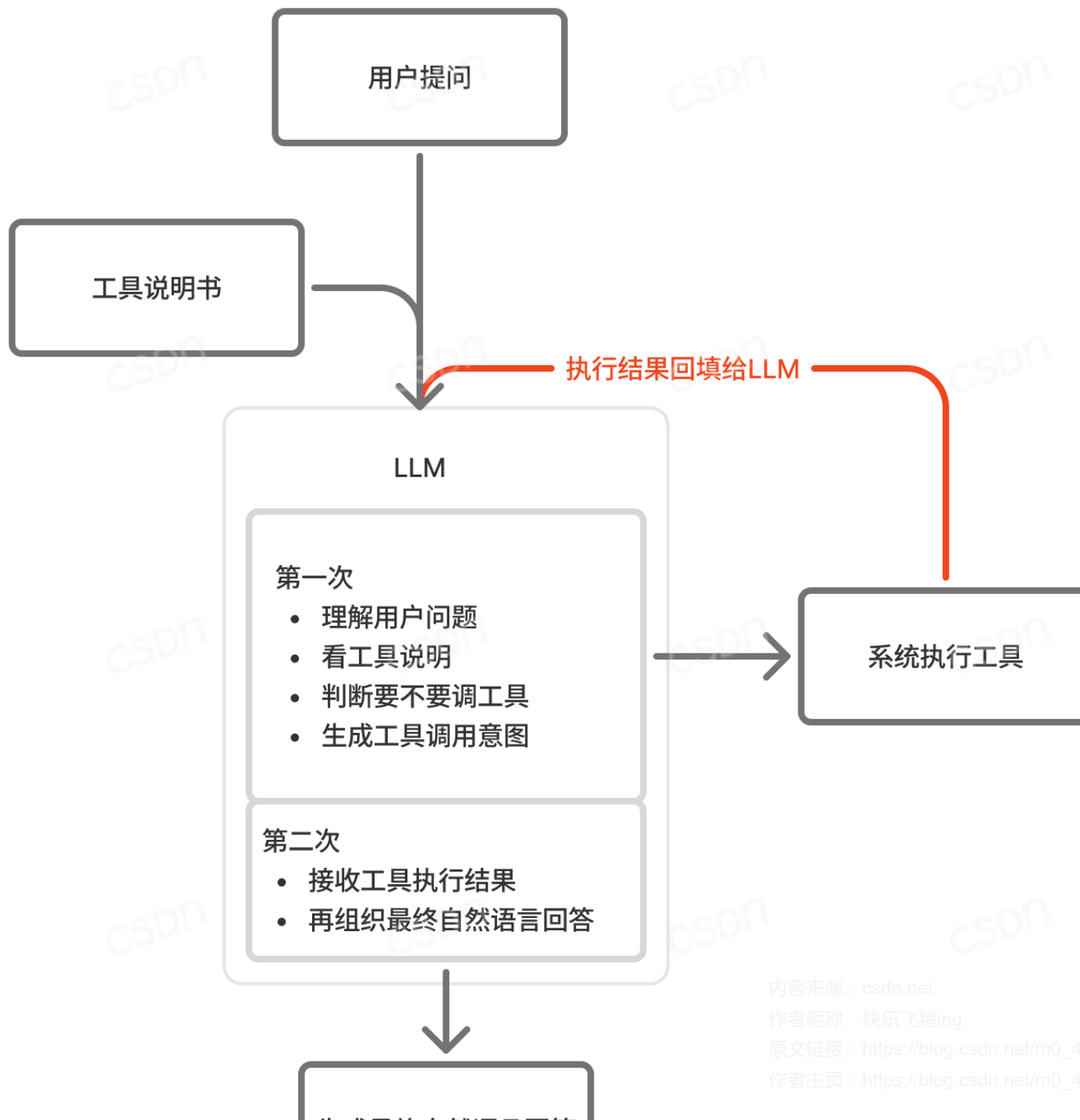
也就是说，Function Call 的本质不是“模型自己执行工具”，而是：**模型负责决策，系统负责执行，结果再返回给模型继续组织答案。**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060



内容来源: csdn.net
作者昵称: 快乐飞猪ing
原文链接: https://blog.csdn.net/m0_46279060/article/details/159312865
作者主页: https://blog.csdn.net/m0_46279060

对 Function Call 最核心的理解是：**用户问题和工具说明会一起给到 LLM，LLM 第一次负责判断是否需要调用工具，并生成工具调用意图；系统再根据这个意图真正执行工具；执行结果返回后，LLM 第二次再基于结果组织最终自然语言回答。**

一、根据图片逐步拆解

1、用户提问：流程的起点

Function Call 的起点，永远是由用户先提出一个自然语言问题。

比如：

- 帮我查一下今天成都的天气
- 帮我看看我的订单现在到哪了
- 帮我发一封邮件给领导
- 帮我查一下这个产品库存还有多少

这些问题有一个共同点：用户说的是“需求”，不是“API 调用命令”。

用户不会告诉模型：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

- 调哪个接口
- 参数怎么写
- 用哪个工具
- 返回值怎么处理

这些都需要后面的系统和模型协同完成。

所以在第一步里，用户只是提出了一个自然语言层面的任务需求。

2、工具说明书：模型为什么会知道有哪些工具可以调用

这是很多初学者最容易困惑的地方：

- 大模型怎么会知道自己可以调天气工具、查订单工具或者发邮件工具？

答案是：因为**系统会提前把可用工具的信息告诉它**。也就是说，在 Function Call 流程里，送进 LLM 的不只是用户问题，通常还会包括一份“工具说明书”。

这份说明里一般会包含：

- 工具名称
- 工具作用
- 参数说明
- 参数类型

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

- 必填字段

比如一个天气工具，系统可能会告诉模型：

- 工具名：get_weather
- 功能：查询指定城市、指定日期的天气
- 参数：city、date

这样模型才知道：哦，原来我现在手上有一个可以查天气的工具。所以模型不是“自己发现工具”，而是**系统在当前上下文里把工具能力显式暴露给它**。

3、LLM 第一次工作：理解问题、看工具说明、判断是否调用工具

这一步是 Function Call 的第一次核心推理。

模型第一次拿到的信息通常包括两部分：

- 用户问题
- 工具说明

它要做的事情包括：

- 第一，理解用户问题。比如用户问的是天气、订单、邮件，还是普通问答。
- 第二，看工具说明。看看当前有哪些工具可用，每个工具分别能干什么。
- 第三，判断要不要调用工具

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

```
1 | 注意了，不是所有问题都要调工具。有些问题模型自己就能答，比如：
2 |
3 | - 什么是函数调用？
4 | - RAG 和 Function Call 有什么区别？
5 |
6 |
7 | 这些就不一定需要工具。但像“今天成都天气怎么样”这种实时问题，
8 | 模型就会判断：这个问题我不能只靠自己的知识回答，需要调工具。
```

- 第四，生成工具调用意图

模型在这一步还没有真正执行工具。它只是生成一段**结构化的调用意图**，比如：我要调用哪个工具？需要传什么参数？

所以这一步更准确地说，不是“模型执行工具”，而是：**模型先生成一份“工具调用请求”**。

4、系统执行工具：真正动手的不是 LLM，而是外部系统

当 LLM 生成了**工具调用**意图之后，接下来真正执行工具的并不是模型，而是外部系统。

这可能是：

- 后端服务
- Agent 框架
- 中间调度层
- 应用程序本身

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

也就是说，系统拿到模型生成的调用请求后，才会真的去：

- 请求天气 API
- 查询订单数据库
- 调用邮件发送服务
- 访问地图或搜索接口

所以如果要用一句话总结这里的分工，那就是：

- LLM 负责决定“做什么”，系统负责真正“去做”。

这一点非常关键，因为它正是 **Function Call** 和普通自然语言生成的本质区别之一。

5、执行结果回填给 LLM：工具执行完，还没有结束

这是另一个很容易被忽略的步骤。

很多人以为系统执行完工具后，流程就结束了。

但实际上，在大多数 Function Call 场景里，**工具结果还需要再回填给 LLM。**

比如天气工具执行完之后，系统可能拿到了这样一份结构化结果：

- 城市：成都
- 日期：今天
- 天气：阴天

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

- 气温：18°C~24°C
- 空气质量：良

这份结果本身更像是机器可读的原始数据，还不是最终给用户看的自然语言回答。

所以这时候系统会把这份执行结果再交回给 LLM。这一步的意义是：

- 让模型基于真实工具结果，再进行第二次组织和表达。

6、LLM 第二次工作：接收结果，再组织最终自然语言回答

当工具结果返回给模型之后，LLM 就开始第二次工作。

这一次，它不再主要负责“判断要不要调工具”，而是负责：

- 理解工具执行结果
- 提取对用户最有用的信息
- 组织成自然语言
- 生成最终回答

还是以天气为例，工具返回的是[结构化数据](#)，但用户最终看到的可能是：

- 今天成都阴天，气温 18°C 到 24°C，空气质量良，适合外出，建议带一件薄外套。

这时候你就会发现：用户最终看到的是一句很自然的话，但它其实不是模型“凭空想出来”的，而是模型在第二次拿到工具结果后，再整理出来的。

内容来源：csdn.net

作者昵称：快乐飞猪ing

文章地址：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

7、生成最终自然语言回答：用户看到的只是最后一层

从用户角度看，整个过程可能只是一瞬间：

- 提一个问题
- 得到一个答案

但从系统内部看，其实已经走完了一整条链路：

- 用户提问
- 工具说明输入给模型
- 模型第一次判断并生成调用意图
- 系统执行工具
- 工具结果回填给模型
- 模型第二次组织回答
- 输出最终自然语言结果

所以最终回答，其实只是这条完整链路的最后一个环节。

二、用“查天气”案例走一遍完整流程

用户提问：

帮我查一下今天成都的天气。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

第一步：用户提问

系统收到一个自然语言需求。

第二步：工具说明一起给到 LLM

系统同时告诉模型：

- 你现在可以使用一个天气工具
- 工具名叫 get_weather
- 输入参数包括 city 和 date

第三步：LLM 第一次工作

模型理解问题后判断：

- 这是一个实时信息问题
- 不能只靠自己已有知识回答
- 当前正好有天气工具可用
- 所以应该调用 get_weather
- 参数是：city=成都，date=今天

第四步：系统执行工具

系统根据模型生成的调用意图，真正去请求天气接口。

第五步：工具执行结果返回

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060

系统得到天气结果：

- 阴天
- 18°C~24°C
- 空气质量良

第六步：结果回填给 LLM

系统把这份结构化结果再交回给模型。

第七步：LLM 第二次工作

模型基于返回结果，把它组织成用户能直接看懂的话：

今天成都阴天，气温 18°C 到 24°C，空气质量良，适合外出，建议带一件薄外套。

到这里，一个完整的 Function Call 流程才算真正结束。

三、思考题：

大模型到底是怎么知道有哪些工具可以调用？又是怎么选中正确工具的？

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159312865

作者主页：https://blog.csdn.net/m0_46279060